

Kapitel 9. Avancerade datatyper

Innehållsförteckning

[Pekare](#)[Vektorer](#)[Vektorer som pekare](#)[Flerdimensionella vektorer](#)[Sammansatta datatyper](#)[Enumereringar](#)[Definiera egna datatyper](#)

Detta kapitel behandlar mera avancerade och sammansatta datatyper som används i C++. Pekare introduceras även.

Pekare

Vi har så här långt undvikit att använda en av de fundamentala byggstenarna i klassisk C (och C++), nämligen *pekare*. Då vi använt variabler tidigare har vi använt variablerna direkt, eller så kopior av dessa. Referensparametrar påminner till en viss mån om pekare (se [avsnittet Parametertyper i Kapitel 7](#)), eftersom man med hjälp av dessa kan i en funktion ändra på värdet av en variabel så att det även "syns" i ursprungsvariabeln. Med hjälp av pekare kan man åstadkomma samma funktionalitet. En pekare är en datatyp som innehåller *minnesadressen* för en variabel av någon typ. Med hjälp av denna adress kan man direkt manipulera variabeln.

Förståelse av pekare innebär att man måste förstå på vilket sätt data lagras i minnet, att varje variabel finns på en viss minnesadress, samt att variabelns värde kan manipuleras via denna minnesadress (om den är känd). För att få minnesadressen för en variabel skall man använda *adressoperatoren* &. Följande exempel skriver ut adresserna på två variabler:

```
#include <iostream>

int main () {
    int Tal;
    double AnnatTal;

    // skriv ut adressen på talen
    cout << "Tal har adressen      : " << &Tal << endl;
    cout << "AnnatTal har adressen: " << &AnnatTal << endl;
}
```

Programmet ger på den använda datorn ut nedanstående utskrift. Notera att dessa värden troligtvis ändras mellan varje körning av programmet.

```
% Pointers1
Tal har adressen      : 0xbffff854
AnnatTal har adressen: 0xbffff848
%
```

Adresserna skrivs ut i hexadecimal bas. För att deklarera en pekare använder man denna allmänna

form:

```
datatyp * variabelnamn;
```

Man definierar alltså pekare att peka på en viss typ av data. En asterisk * indikerar att det är frågan om en pekare.

Varning

Blanda inte ihop adressoperatoren & med den binära operatoren & eller & då det används för att indikera en referensparameter. Se även upp med * då det kan användas som både pekardefinition och multiplikationsoperator.

Exempel på pekardefinitioner är:

```
int * TalPekare1;
string * TextPekare;
float * FlyttalsP1, * FlyttalsP2;
```

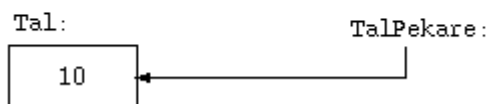
för att tilldela en pekare adressen på en variabel använder man följande allmänna definition:

```
datatyp variabelnamn;
datatyp * pekarnamn;
pekarnamn = &variabelnamn;
```

Adressen som tilldelas en pekare måste vara adressen på en variabel av samma datatyp som pekaren är definierad att peka på. Det är således illegalt att tilldela en `float *` värdet på en `string`. Schematiskt ser det ut som nedan då en tilldelning gjorts:

Figur 9-1. Schematisk bild över pekare

```
int Tal = 10;
int * TalPekare = &Tal;
```



Följande lilla exempel kan användas för att visa vilken adress en pekare pekar på:

```
#include <iostream>

int main () {

    double Tal;
    double * TalPekare = &Tal;

    // skriv ut adressen på talet och pekarens adress
    cout << "Tal har adressen          : " << &Tal << endl;
    cout << "TalPekare har adressen : " << TalPekare << endl;
}
```

På den använda maskinen ger programmet följande utskrift:

```
% Pointers2
```

```
Tal har adressen      : 0xbffff850
TalPekare har adressen: 0xbffff850
%
```

Man kan således enkelt skriva ut både pekare och minnesadresser. Notera att en pekare som inte initialiserats att peka på en viss minnesadress (variabel) har ett odefinierat värde. Om man inte direkt skall använda en pekare kan det löna sig att tilldela den värdet 0 för att initialisera den till ett känt värde. Detta värde är sedan enkelt att känna igen i en debugger om någonting går fel. Initialisera t.ex. så här:

```
double Tal;
double * TalPekare = 0;
...
// långt senare tilldelas den ett värde
TalPekare = &Tal;
```

Avreferera pekare

Pekare är inte till mycken glädje om man endast kan använda dem till att skriva ut adresser. Som redan tidigare nämnts så kan man även accessera det data som en pekare refererar till. Detta görs genom att använda *avrefereringsoperatoren* *. Symbolen * har många användningsområden. Allmänt ser det ut så här:

```
*pekare;
```

Ett exempel klargör vad det är frågan om:

```
#include <iostream>

int main () {

    double Tal;
    double * TalPekare = &Tal;

    // skriv ut adressen på talet och pekarens adress
    cout << "Tal har värdet      : " << Tal << endl;
    cout << "*TalPekare har värdet: " << *TalPekare << endl;
}
```

På den använda maskinen ger programmet följande utskrift:

```
% Pointers3
Tal har värdet      : 10
*TalPekare har värdet: 10
%
```

Både Tal och *TalPekare gav samma värde. Så länge som pekaren pekar på Tal kommer det alltid att vara så. Man kan använda en *avrefererad* pekare i alla sammanhang som man kan använda variabeln som pekaren pekar på. Man kan säga att en avrefererad pekare har samma datatyp som datatypen den pekar på. I exemplet ovan skulle Talpekare kunna användas i alla sammanhang där double kan användas. Man kan t.ex. tilldela eller jämföra en avrefererad pekare.

Varning

Pekare i oförsiktiga händer är ett enkelt sätt att introducera allvarliga fel som kan vara svåra att
--

hitta! En alltför stor del av programfel i C++ förorsakas av felaktig hantering av pekare.

Pekare som parametrar

Pekare kan förstås även skickas som parametrar till funktioner och även returneras som funktionsvärden. Genom att skicka pekare till en funktion kan den mottagande funktionen ändra på det värde pekaren pekar på. Samma effekt kan nås om man istället för pekare använder referensparametrar som specificeras med tecknet `&`. De båda påminner mycket om varandra, och största skillnaden är hur man refererar till dem. Pekare måste avrefereras med referensparametrar inte behöver göra det. Om man vill skicka parametrar som skall kunna modifieras till funktioner är det egalt vilket sätt som används. Ett exempel på en funktion som läser in ett värde i en funktion och skriver ut det i en annan:

```
#include <iostream>

void in (int * Tal) {
    // läs in ett tal
    cout << "Ge in ett tal: ";
    cin >> *Tal;
}

void ut (int * Tal) {
    // skriv ut vårt tal
    cout << "Talet är: " << *Tal << endl;
}

int main () {

    int Tal;

    // läs in och skriv sedan ut
    in ( &Tal );
    ut ( &Tal );
}
```

Pekare kan alltså hanteras som vilken annan datatyp som helst då det gäller att skicka dem som parametrar. Notera att om en pekare skickas som parameter kan endast värdet på det som pekaren pekar på ändras, *inte* själva adressen den pekar på. I exemplet ovan kan t.ex. `in()` inte ändra `Tal` att peka på en annan variabel. Minnesadressen som skickats till `in()` och `ut()` kan inte ändras, den är "kopierad" från `main()`. Vill man kunna ändra den minnesadress en pekare pekar på i t.ex. `in()` måste en *pekare till en pekare* skickas som parameter. Detta skrivs med två `**`. Det är dock sällan man behöver använda pekare till pekare.

Konstanta pekare

Det går även definiera pekare som konstanter på olika sätt. Man kanske vill skicka en pekare till någon speciell datatyp som en parameter till en funktion, men inte vill att mottagaren skall kunna ändra det pekaren pekar på. I så fall kan man skapa konstanta pekare och använda `const` som extra parameterbeteckning i den mottagande funktionen. En konstant pekare kan deklarerars t.ex. på följande sätt:

```
const int Tal = 100;
const int * KonstantPekare = &Tal;
```

En funktion som tar en konstant som parameter pekare kan se ut på följande sätt:

```
void berakna (const int * Tal) {  
    ...  
}
```

Man kan även definiera en pekare själv som konstant, trots att det den pekar på inte är det. I så fall kan man använda följande form:

```
int Tal = 100;  
int * const KonstantPekare = &Tal;
```

I detta fall kan man nog ändra värdet på det som `KonstantPekare` pekar på, men däremot *inte* på pekaren själv. Den är alltså "låst" att peka på en och samma minnesadress. Med en `const int *` kan man nog ändra på själva pekarens värde, d.v.s. vilken minnesadress den pekar på, men däremot inte värdet på det den pekar på. En tredje form kombinerar dessa båda formerna och skapar en konstant pekare där man inte kan ändra på vare sig pekaren eller det pekaren pekar på:

```
int Tal = 100;  
const int * const KonstantPekare = &Tal;
```

Speciellt den första formen med pekare som inte kan ändra på det de pekar på används ofta då man skickar data som funktionsparametrar. Numera verkar man dock även i de fallen gå över till att använda referensparametrar istället, och definiera parametrarna som `const` istället. Genom att använda referensparametrar behöver man inte komma ihåg vilka funktioner som skall ha pekare och vilka funktioner som skall ha normala parametrar. Använder man referensparametrar med `const` på lämpliga ställen är funktionerna lättare att använda ur anroparens synpunkt.

[Förutgående](#)[Hem](#)[Nästa](#)Utskrift till `cerr` och `clog`

Vektorer